

# Rapport Complet sur l'Expressivité du Langage PlanningSpec

Analyse technique du DSL de planification

18 avril 2026

## Table des matières

|          |                                                               |          |
|----------|---------------------------------------------------------------|----------|
| <b>1</b> | <b>Introduction</b>                                           | <b>3</b> |
| <b>2</b> | <b>Périmètre d'expressivité</b>                               | <b>3</b> |
| 2.1      | Objets modélisés . . . . .                                    | 3        |
| 2.2      | Positionnement . . . . .                                      | 3        |
| <b>3</b> | <b>Modèle mathématique global</b>                             | <b>3</b> |
| 3.1      | Ensembles et paramètres . . . . .                             | 3        |
| 3.2      | Variables de décision . . . . .                               | 4        |
| 3.3      | Contraintes structurelles générées systématiquement . . . . . | 4        |
| <b>4</b> | <b>Contraintes du DSL : sémantique et traduction MiniZinc</b> | <b>4</b> |
| 4.1      | cardinality_per_activity . . . . .                            | 4        |
| 4.2      | resource_exclusivity . . . . .                                | 5        |
| 4.3      | fixed_assignment . . . . .                                    | 5        |
| 4.4      | forbidden_assignment . . . . .                                | 5        |
| 4.5      | temporal_precedence . . . . .                                 | 6        |
| 4.6      | time_window . . . . .                                         | 6        |
| 4.7      | mandatory_roles . . . . .                                     | 6        |
| 4.8      | instance_precedence . . . . .                                 | 6        |
| 4.9      | required_resource . . . . .                                   | 6        |
| <b>5</b> | <b>Préférences : modélisation souple et fonction objectif</b> | <b>7</b> |
| 5.1      | Principe . . . . .                                            | 7        |
| 5.2      | avoid_participation_on_date . . . . .                         | 7        |
| 5.3      | max_per_scope . . . . .                                       | 7        |
| 5.4      | preferred_resource . . . . .                                  | 8        |
| <b>6</b> | <b>Lien explicite entre DSL et fragments MiniZinc</b>         | <b>8</b> |
| 6.1      | Exemple de solve . . . . .                                    | 8        |
| 6.2      | Lecture logique . . . . .                                     | 8        |

|          |                                                             |          |
|----------|-------------------------------------------------------------|----------|
| <b>7</b> | <b>Évaluation de robustesse et d'expressivité</b>           | <b>8</b> |
| 7.1      | Forces . . . . .                                            | 8        |
| 7.2      | Limites . . . . .                                           | 8        |
| 7.3      | Conclusion . . . . .                                        | 9        |
| <b>8</b> | <b>Annexe A : Récapitulatif synthétique des traductions</b> | <b>9</b> |

# 1 Introduction

Ce document présente une analyse complète de l’expressivité du langage **PlanningSpec**, conçu pour modéliser des problèmes de planification à ressources contraintes. L’objectif est double :

- expliquer précisément ce que le langage peut représenter ;
- documenter, pour chaque contrainte et préférence, la traduction MiniZinc et la logique mathématique sous-jacente.

La chaîne technique étudiée est :

DSL **PlanningSpec**  $\rightarrow$  AST **Langium**  $\rightarrow$  Générateur **MiniZinc**  $\rightarrow$  Solveur

## 2 Périmètre d’expressivité

### 2.1 Objets modélisés

Le langage couvre les éléments suivants :

- **Temps discret** : jours et créneaux par jour ;
- **Activités** : types d’activité, nombre d’instances, durée ;
- **Ressources** : types de ressources et instances nommées ;
- **Rôles** : rôles applicables à une activité et type de ressource autorisé ;
- **Contraintes dures** : obligatoires pour la faisabilité ;
- **Préférences souples** : pénalisées dans une fonction objectif.

### 2.2 Positionnement

Le DSL est expressif pour des variantes de planification discrète proches de RCPSP/-JSSP simplifiés :

- précédences d’activités,
- fenêtres temporelles,
- contraintes de capacité/exclusivité sur ressources,
- affectations imposées/interdites,
- optimisation de qualité par pénalité.

## 3 Modèle mathématique global

### 3.1 Ensembles et paramètres

Soient :

|               |                                                 |
|---------------|-------------------------------------------------|
| $\mathcal{A}$ | : ensemble des instances d’activités (ACT_INST) |
| $\mathcal{R}$ | : ensemble des ressources (RESOURCE)            |
| $\mathcal{K}$ | : ensemble des rôles (ROLE)                     |
| $H$           | : horizon total en slots (TOTAL_SLOTS)          |
| $d_i$         | : durée de l’activité $i \in \mathcal{A}$       |

Sous-ensembles typés :

$$\begin{array}{ll} \mathcal{R}_\tau \subseteq \mathcal{R} & : \text{ressources du type } \tau \\ \mathcal{A}(a) \subseteq \mathcal{A} & : \text{instances du type d'activité } a \end{array}$$

### 3.2 Variables de décision

$$\begin{array}{ll} s_i \in \{1, \dots, H\} & : \text{slot de début de l'activité } i \\ x_{i,r} \in \{0, 1\} & : \text{la ressource } r \text{ est affectée à } i \\ y_{i,k,r} \in \{0, 1\} & : \text{la ressource } r \text{ tient le rôle } k \text{ dans } i \end{array}$$

### 3.3 Contraintes structurelles générées systématiquement

**Non dépassement journalier**

$$\forall i \in \mathcal{A} : ((s_i - 1) \bmod SLOTS\_PER\_DAY) + d_i \leq SLOTS\_PER\_DAY$$

**Non-chevauchement par ressource**

$$\forall r \in \mathcal{R}, \forall i < j : (x_{i,r} \wedge x_{j,r}) \Rightarrow (s_i + d_i \leq s_j \vee s_j + d_j \leq s_i)$$

**Compatibilité rôles/ressources** Pour chaque activité et rôle :

si le rôle n'est pas applicable :  $y_{i,k,r} = 0$

$$y_{i,k,r} = 1 \Rightarrow x_{i,r} = 1$$

$$\sum_k y_{i,k,r} \leq 1 \quad (\text{une ressource ne porte pas deux rôles sur la même activité})$$

## 4 Contraintes du DSL : sémantique et traduction MiniZinc

### 4.1 cardinality\_per\_activity

**Sémantique**

Contrôle le nombre d'affectations par instance d'une activité, soit :

- sur un `target` (type de ressource),
- sur un `role`,
- ou le cas `target="slot"`.

**Traduction mathématique**

**Cible type de ressource  $\tau$**

$$\forall i \in \mathcal{A}(a) : m \leq \sum_{r \in \mathcal{R}_\tau} x_{i,r} \leq M$$

**Cible rôle  $k$**

$$\forall i \in \mathcal{A}(a) : \quad m \leq \sum_{r \in \mathcal{R}_{type(k)}} y_{i,k,r} \leq M$$

**Cible slot** Le modèle ayant exactement un début  $s_i$  par instance, la traduction vérifie :

$$m \leq 1 \leq M$$

## 4.2 resource\_exclusivity

**Sémantique**

Limiter l'utilisation d'une ressource d'un type donné pour une activité donnée selon une portée **slot** ou **day**.

**Traduction**

**Scope slot**

$$\forall r \in \mathcal{R}_\tau, \forall t \in \{1, \dots, H\} : \quad \sum_{i \in \mathcal{A}(a)} \mathbf{1}[x_{i,r} = 1 \wedge s_i \leq t < s_i + d_i] \leq M$$

**Scope day**

$$\forall r \in \mathcal{R}_\tau, \forall q \in \{1, \dots, NUM\_DAYS\} : \quad \sum_{i \in \mathcal{A}(a)} \mathbf{1}[x_{i,r} = 1 \wedge dayOf(s_i) = q] \leq M$$

## 4.3 fixed\_assignment

**Sémantique**

Impose qu'une ressource donnée soit affectée à une instance d'activité (et à un rôle quand défini).

**Traduction**

$$x_{i,r} = 1, \quad y_{i,k,r} = 1 \text{ (si rôles actifs)}$$

## 4.4 forbidden\_assignment

**Sémantique**

Interdit une affectation explicite d'une ressource à un rôle pour une instance.

**Traduction**

$$y_{i,k,r} = 0$$

## 4.5 temporal\_precedence

### Sémantique

Toute instance de l'activité `beforeActivity` doit se terminer avant toute instance de `afterActivity`.

### Traduction

$$\forall i \in \mathcal{A}(a_{before}), \forall j \in \mathcal{A}(a_{after}) : s_i + d_i \leq s_j$$

## 4.6 time\_window

### Sémantique

L'instance doit démarrer et finir dans une fenêtre  $[L, U]$ .

### Traduction

$$s_i \geq L, \quad s_i + d_i - 1 \leq U$$

## 4.7 mandatory\_roles

### Sémantique

Chaque rôle applicable d'une activité doit être rempli au moins une fois par instance.

### Traduction

$$\forall i \in \mathcal{A}(a), \forall k \text{ applicable} : \sum_{r \in \mathcal{R}} y_{i,k,r} \geq 1$$

## 4.8 instance\_precedence

### Sémantique

Précédence stricte entre deux instances nommées.

### Traduction

$$s_{i_{before}} + d_{i_{before}} \leq s_{i_{after}}$$

## 4.9 required\_resource

### Sémantique

Une ressource donnée doit participer à une instance donnée.

## Traduction

$$x_{i,r} = 1$$

## 5 Préférences : modélisation souple et fonction objectif

### 5.1 Principe

Les préférences ne rendent pas le modèle infaisable. Elles ajoutent des pénalités :

$$penalty = \sum_{\ell} P_{\ell}$$

La résolution est :

- `min penalty` s'il existe des préférences ;
- `satisfy` sinon.

### 5.2 `avoid_participation_on_date`

#### Sémantique

Éviter qu'une ressource participe à au moins une activité sur un jour donné.

#### Traduction

Si la date correspond au jour  $q$  :

$$P = w \cdot \mathbf{1} \left[ \sum_{i \in \mathcal{A}} \mathbf{1}[x_{i,r} = 1 \wedge dayOf(s_i) = q] > 0 \right]$$

### 5.3 `max_per_scope`

#### Sémantique

Préférence de plafonnement souple du nombre d'affectations au-delà d'un maximum.

#### Traduction (`scope slot`)

$$P = \sum_{r \in \mathcal{R}_{\tau}} \sum_{t=1}^H w \cdot \max \left( 0, \sum_{i \in \mathcal{A}(a)} \mathbf{1}[x_{i,r} = 1 \wedge s_i \leq t < s_i + d_i] - M \right)$$

#### Traduction (`scope day`)

$$P = \sum_{r \in \mathcal{R}_{\tau}} \sum_{q=1}^{NUM\_DAYS} w \cdot \max \left( 0, \sum_{i \in \mathcal{A}(a)} \mathbf{1}[x_{i,r} = 1 \wedge dayOf(s_i) = q] - M \right)$$

## 5.4 preferred\_resource

### Sémantique

Favoriser une ressource donnée pour un couple (activité, rôle) ou une activité sans rôles.

### Traduction

$$P = \begin{cases} w \cdot \mathbf{1}[y_{i,k,r} = 0] & \text{si rôles actifs} \\ w \cdot \mathbf{1}[x_{i,r} = 0] & \text{sinon} \end{cases}$$

## 6 Lien explicite entre DSL et fragments MiniZinc

### 6.1 Exemple de solve

```
solve :: int_search([start_time[i] | i in ACT_INST],
                    first_fail, indomain_min, complete)
    minimize penalty;
```

### 6.2 Lecture logique

- `first_fail` : instancie d'abord les variables les plus contraintes ;
- `indomain_min` : essaye les valeurs basses en premier ;
- `complete` : recherche complète.

## 7 Évaluation de robustesse et d'expressivité

### 7.1 Forces

- séparation nette dur/souple ;
- couverture riche des contraintes métiers usuelles ;
- traduction MiniZinc structurée et traçable ;
- langage éditable facilement (syntaxe JSON-like).

### 7.2 Limites

- `temporal_precedence` est actuellement de type all-to-all ;
- `target="slot"` dans `cardinality_per_activity` est une vérification de cohérence plutôt qu'une décision ;
- pas d'objectif `makespan` activé par défaut (choix de performance) ;
- validation sémantique encore extensible (cohérence globale, cycles, etc.).



### 7.3 Conclusion

Le langage est aujourd'hui suffisamment robuste pour des scénarios de planification discrète contraints orientés production, tout en restant extensible vers des formulations plus avancées (multi-objectif, critères hiérarchiques, variantes RCPSP/JSSP plus strictes).

## 8 Annexe A : Récapitulatif synthétique des traductions

| Élément DSL                 | Traduction mathématique principale                             |
|-----------------------------|----------------------------------------------------------------|
| cardinality_per_activity    | $n \leq \sum x_{i,r} \leq M$ ou $m \leq \sum y_{i,k,r} \leq M$ |
| resource_exclusivity        | $\sum \mathbf{1}[\text{actif au slot/jour}] \leq M$            |
| fixed_assignment            | $x_{i,r} = 1$ , éventuellement $y_{i,k,r} = 1$                 |
| forbidden_assignment        | $y_{i,k,r} = 0$                                                |
| temporal_precedence         | $s_i + d_i \leq s_j$ pour $i \in A_{before}, j \in A_{after}$  |
| time_window                 | $L \leq s_i, s_i + d_i - 1 \leq U$                             |
| mandatory_roles             | $\sum_r y_{i,k,r} \geq 1$                                      |
| instance_precedence         | $s_{i_b} + d_{i_b} \leq s_{i_a}$                               |
| required_resource           | $x_{i,r} = 1$                                                  |
| avoid_participation_on_date | $\neg \exists i : x_{i,r} = 1 \wedge dayOf(s_i) = q$           |
| max_per_scope               | $w \cdot \max(0, \text{charge} - M)$ agrégé par slot/jour      |
| preferred_resource          | $w \cdot \mathbf{1}[\text{ressource préférée non choisie}]$    |